Switched Local Area Networks

Lecture 28

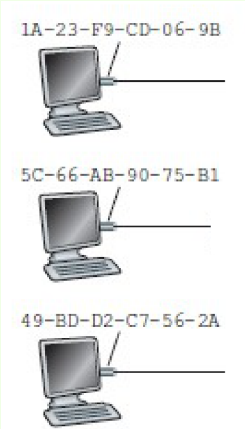
Computer Networks and Distributed Programming

Stations get names

- On a shared channel we let stations contend, collide and back off — and never asked whom a frame was for.
- A real local network has to say it. The link layer does so with names of its own: MAC addresses.
- Two promises come due with them: the ARP we named when hosts got their addresses, and the Ethernet frame we kept postponing.
- Today: how interfaces are named, how a switch uses the names, how ARP finds them, and what an Ethernet frame carries.
- At the end, your chat once more — captured one layer lower than the last time.

A name for every interface

- At the link layer a frame is addressed to an interface, by its MAC address — also called LAN or physical address.
- Six bytes, 2^{48} possible values, written in hexadecimal: `1A-23-F9-CD-06-9B`, or with colons.
- One per adapter: a router with three interfaces has three MAC addresses, as it has three IP addresses.
- Local: it names the next node on the same link, and never travels past the first router.



Reading your own MAC address

```
$ ip -br link
lo        UNKNOWN   00:00:00:00:00:00  <LOOPBACK,UP,LOWER_UP>
enp3s0    UP        3c:52:82:4a:1f:07  <BROADCAST,MULTICAST,UP,LOWER_UP>
wlp2s0    UP        f2:1d:6c:93:a0:5e  <BROADCAST,MULTICAST,UP,LOWER_UP>
$ ip link show enp3s0
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP ...
    link/ether 3c:52:82:4a:1f:07 brd ff:ff:ff:ff:ff:ff
```

- Every Ethernet and WiFi interface has one; `lo` has six zero bytes, because nothing ever leaves on it.
- `brd` is the broadcast address of the link: 48 ones. We will need it in a moment.
- The first three bytes of the wired address identify its maker. The WiFi one does not look like any maker at all — next frame.

Who hands out MAC addresses

- IEEE sells a manufacturer 2^{24} addresses by fixing the first 24 bits, the OUI; the manufacturer numbers the other 24.
- No two adapters share an address, and the space is flat: nothing in it says where the adapter is.
- An IP address is hierarchical and changes when the host moves; a MAC address goes wherever the adapter goes.
- Two bits of the first byte: the lowest says unicast or group; the next says universal or locally assigned.
- Meant to be burnt in, it can be changed in software: phones and laptops pick a random local one per WiFi network.

The two addresses of the previous frame

`3c` = 0011 1100: unicast, universal — a manufacturer's. `f2` = 1111 0010: unicast, local — made up by the laptop.

Why an interface needs two addresses

- Why not use the IP address on the LAN as well? Because a LAN is built for any network layer, and IP is only one of them.
- An adapter that knew IP addresses would need reconfiguring every time it moved or booted. A MAC address comes with the hardware.
- And an adapter with no address of its own would pass every frame up, interrupting the host for traffic meant for others.
- So each layer names things its own way: host names above, IP addresses at the network layer, MAC addresses on the link.
- The layers stay independent. The price is a translation between the last two — which is what ARP is for.

Accept, reject, or broadcast

- Sending: *A* writes *B*'s MAC address in the destination field and transmits the frame into the LAN.
- Receiving: every adapter compares the destination with its own address: match, the datagram goes up; no match, dropped.
- Adapters hear many frames that are not theirs. The address exists to discard them without disturbing the host.
- Broadcast: destination `FF-FF-FF-FF-FF-FF`: every adapter accepts the frame and passes it up.
- Promiscuous mode: the adapter stops filtering and hands up everything it sees. It is what Wireshark asks of it.

A MAC address for a multicast group

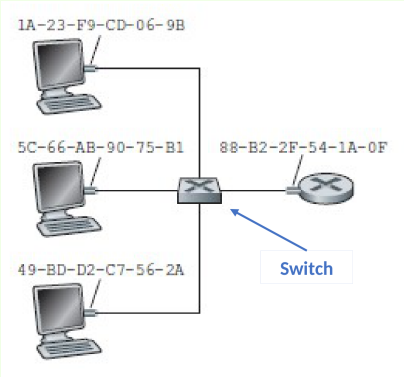
Your multicast peer sends to 239.1.2.3. There is no host to ask for its MAC address. What goes in the destination field?

An address computed from the group: `01-00-5E`, then the low 23 bits of the IP address — here `01-00-5E-01-02-03`.

- The first byte ends in a 1: a group address, the bit we just read. No ARP is involved, and none could be.
- A group address has 28 bits, the mapping keeps 23: 32 groups share every multicast MAC address.
- The adapter accepts all 32; the kernel discards the groups nobody joined. That is the filtering left to each kernel when a switch floods the group everywhere.

Switches

- The link-layer counterpart of a router: it forwards frames onto outgoing links, reading MAC addresses only.
- No routing algorithm: it forwards from a switch table of addresses and ports, which fills itself.
- Transparent: hosts address frames to each other, never to the switch — which has no address in the picture.
- Output ports have buffers, as a router's do: two frames may want the same port at once.



Why a switch ends collisions

- A hub works on bits: whatever enters one port is repeated on all the others. Every station is in one collision domain.
- A hub star is a shared cable folded into a box, and it still needs CSMA/CD.
- A switch takes in whole frames and sends each one out on one port: every port is a collision domain of its own.
- On a full-duplex cable, station and port transmit at the same time without interfering. A collision is impossible.
- This is where CSMA/CD stops. It is still in the standard, and on a switched link it has nothing to do.
- And several pairs of stations talk at once: a switch's capacity is the sum of its ports.

The switch table

Address	Interface	Time
62-FE-F7-11-89-A3	1	9:32
7C-BA-B2-B4-91-10	3	9:36

A frame for DD-DD-DD-DD-DD-DD arrives on interface x:

- No entry: copy it to every interface except x: the switch floods.
- Entry says x: the destination sits on the segment the frame came from — a hub on that port. Filter: drop it.
- Entry says $y \neq x$: put it in the output buffer of y only: forward.
- With a complete table nothing is flooded: every frame leaves only where it must.

Learning by listening

- Every incoming frame: record its source address, the port it came in on, and the time. The sender has just said where it is.
- Aging: an entry not refreshed for a while is deleted, so a machine that moves is found again.
- No configuration, no protocol: the table builds itself from traffic. A switch is plug-and-play.

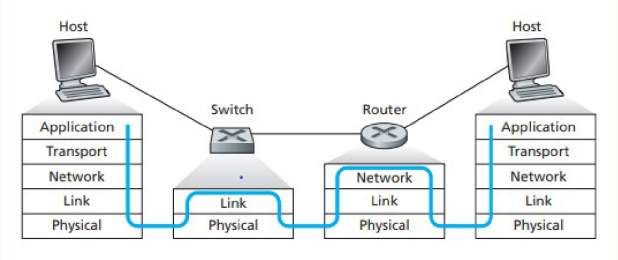
Three frames through an empty switch

Frame	**The switch**	**It learns**
$A \to B$, B unknown	floods it	A is on port 1
$B \to A$, A known	forwards it on port 1 only	B is on port 2
$A \to B$	forwards it on port 2 only	nothing new

Seeing traffic that is not yours

- On a hub every frame reaches every port: one adapter in promiscuous mode reads the whole LAN.
- On a switch it reads what the switch sends it: its own frames, broadcasts, and frames flooded to unknown destinations.
- Switch poisoning: flood the switch with frames from thousands of invented source addresses. The table fills with lies and the switch floods almost everything.
- So a switch makes sniffing harder, not impossible. What must stay private has to be encrypted.
- It is why Wireshark on a lab machine shows your traffic and the broadcasts, and not your neighbour's.

Switch or router



- Both are store-and-forward packet switches: they differ in how high up the stack they look.
- The switch stops at layer 2 and forwards on MAC addresses; the router climbs to layer 3 and forwards on IP addresses.
- Match plus action once more, on another address in another header.

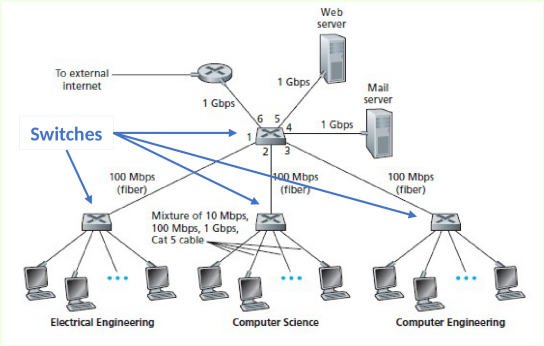
When to use which

	Hubs	Routers	Switches
Traffic isolation	no	yes	yes
Plug and play	yes	no	yes
Optimal routing	no	yes	no

- Switches: fast and plug-and-play; but the topology in use must be a tree, and one broken host can flood everybody with broadcasts.
- Routers: any topology and the best path, and a broadcast stops there; but every interface needs an IP address.
- A few hundred hosts: switches alone. Thousands: switches inside each part, routers between the parts.

Switched LANs

- Real LANs chain several switches: departments, servers, the way out — all at layer 2.
- Fast and plug-and-play: two layers, no routing, nothing to configure.
- Limited in size: flat addresses have no prefix, so every table grows with every host.
- The active topology must be a tree: spare cables may exist, but may not all carry frames. Why?



Two cables between two switches

Two switches are joined by two cables, for redundancy. A host sends a frame to an address neither has learnt. What happens?

Each switch floods it on both cables, receives the other's copies, and floods them again. The frame circulates for ever.

- A switch cannot tell a copy from a new frame: nothing in the frame says it has been seen before.
- And nothing ever kills it. The Ethernet frame has no TTL; IP has one precisely so that a loop ends, as traceroute showed.
- A broadcast does the same on every loop at once: a broadcast storm, and the LAN is down within seconds.

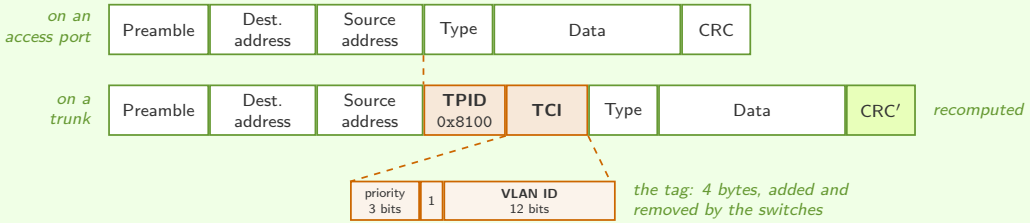
Spanning tree

- Keep the redundant cables, but use a tree of them: one path between any two stations, so no frame can come back.
- The switches exchange messages and elect a root: the one with the lowest identifier, built from its MAC address.
- Each switch keeps its shortest path to the root and turns off the ports that are not on it.
- It never stops running: if a cable of the tree breaks, a switched-off one is brought back and the tree is rebuilt.
- Radia Perlman's algorithm, standardised as IEEE 802.1D. Switches do run one algorithm — not to choose routes, only to avoid loops.

One switch, several LANs

- The departmental tree we have just seen has three weak points.
- No isolation: broadcasts — ARP, DHCP, unknown destinations — cross the whole institution, and anyone sniffing sees them.
- Wasted switches: ten small departments need ten switches, where one with 96 ports would do.
- Moving people: a user who changes department must be recabled to another switch.
- Port-based VLAN: the ports of one switch are split into groups; each group is a separate LAN, and a broadcast domain.
- Between two VLANs traffic goes through a router, as between two LANs — often inside the same box.

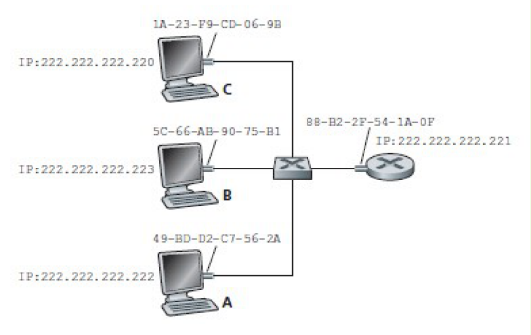
Trunks and the 802.1Q tag



- Two VLAN switches in two buildings: one trunk carries every VLAN, not one cable per VLAN.
- Each frame on the trunk says its VLAN: 12 bits, so at most 4094 VLANs. The CRC covers the tag, so it is recomputed at both ends.
- 0x8100 sits exactly where the type was: to a device that knows nothing of VLANs, the frame carries an unknown protocol.

From IP address to MAC address

- The network layer hands down a datagram and an IP address. The frame needs a MAC address: ARP translates.
- Every interface has an ARP module and an ARP table: IP address, MAC address, and an expiry time.
- Like DNS, it resolves one name into another. Unlike DNS, it only works on the same subnet.



Asking the whole LAN

IP Address	MAC Address	TTL
222.222.222.221	88-B2-2F-54-1A-0F	13:45:00
222.222.222.223	5C-66-AB-90-75-B1	13:52:00

- C (222.222.222.220) has a datagram for A (222.222.222.222), and A is not in its table.
- C sends an ARP request to `FF-FF-FF-FF-FF-FF`: every adapter on the subnet passes it to its ARP module.
- Only the owner of that IP answers, with an ARP reply to C alone. C stores the entry and sends the datagram.

Why is the request broadcast, and the reply not?

An ARP packet

Field	Bytes	Request from *C*	Reply from *A*
hardware, protocol, lengths	6	Ethernet, IPv4, 6, 4	the same
operation	2	1, request	2, reply
sender MAC	6	*C*'s	*A*'s
sender IP	4	222.222.222.220	222.222.222.222
target MAC	6	unknown: zeros	*C*'s
target IP	4	222.222.222.222	222.222.222.220

- Carried straight in an Ethernet frame of type 0x0806: no IP header, no ports.
- It holds addresses of both layers, so it belongs to both and to neither: ARP straddles the network–link boundary.
- The whole packet is 28 bytes. Keep the number for the minimum frame size.

The ARP table on your machine

```
$ ip neigh
192.168.1.1  dev enp3s0 lladdr 10:bd:18:e5:74:80 REACHABLE
192.168.1.17 dev enp3s0 lladdr 3c:52:82:4a:2b:91 STALE
$ sudo ip neigh flush dev enp3s0      forget everything
$ ping -c1 192.168.1.17 > /dev/null   one datagram, so first one ARP
$ ip neigh show 192.168.1.17
192.168.1.17 dev enp3s0 lladdr 3c:52:82:4a:2b:91 REACHABLE
```

- No expiry column but a state: REACHABLE was just confirmed, STALE is still used and rechecked when next needed.
- Either way an entry expires: a host that leaves drops out of every table by itself.
- After the flush the ping had to wait for an ARP exchange. We watch one at the end.
- It is also how `nmap -sn` finds your own subnet: one request per address, hence the MAC addresses.

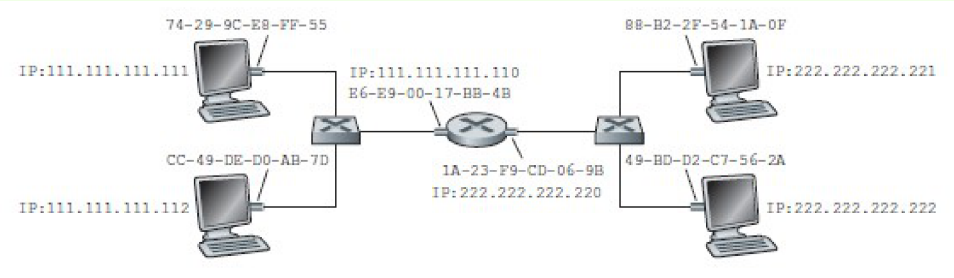
Nobody checks the answer

An ARP reply says that 192.168.1.1 is at some MAC address. Who verifies that it is true?

Nobody. ARP has no authentication, and hosts accept replies they never asked for.

- Unsolicited replies have an honest use: a host that joins announces its own mapping, a gratuitous ARP.
- An attacker announces the gateway's IP with its own MAC. Every host updates its table, and sends it the traffic for outside.
- It forwards everything on, so nothing seems broken: a man in the middle, one of the commonest attacks on a LAN.
- Defences sit outside ARP: a static entry for the gateway, switches that check every reply against what DHCP handed out.

Leaving the subnet



- Two subnets, `111.111.111/24` and `222.222.222/24`, joined by a router with an interface in each.
- Two IP addresses, two MAC addresses, two adapters, and two ARP modules — each resolving only inside its own subnet.

What does a host on the left write on a frame for a host on the right?

Which MAC address goes on the frame?

111.111.111.111 has a datagram for 222.222.222.222. Which destination MAC does it write?

Not the destination's: with `49-BD-D2-C7-56-2A` on it, no adapter on the left matches, and the frame is dropped by all of them.

- The host compares the destination with its own address under the mask: another subnet, so the next hop is the default gateway, 111.111.111.110.
- ARP resolves the gateway, not the destination. The frame carries `E6-E9-00-17-BB-4B`, and the router's adapter takes it.
- The router looks up its forwarding table, picks the interface 222.222.222.220, and builds a new frame — resolved by its own ARP.

What changes at every hop

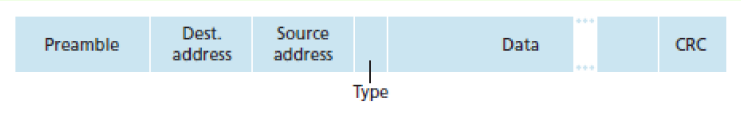
	Frame on the left subnet	**Frame on the right subnet**
Source MAC	74-29-9C-E8-FF-55	1A-23-F9-CD-06-9B
Destination MAC	E6-E9-00-17-BB-4B	49-BD-D2-C7-56-2A
Source IP	111.111.111.111	111.111.111.111
Destination IP	222.222.222.222	222.222.222.222

- The IP addresses are written once, by the sender, and arrive unchanged: they name the two ends of the path.
- The MAC addresses are rewritten by every router: they name the two ends of one link.
- So each hop costs one lookup in an ARP table. Address translation is the exception on the IP side, and it is deliberate.

From a coaxial cable to a switch

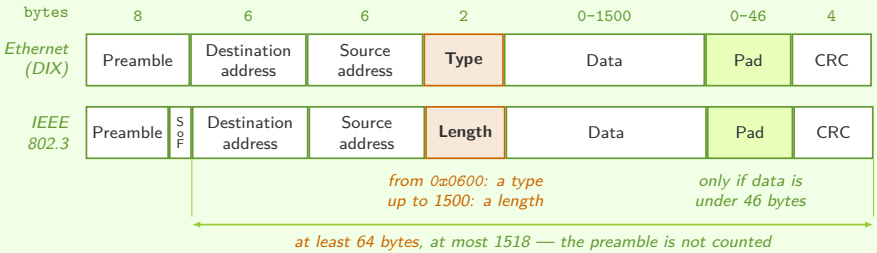
- Bob Metcalfe and David Boggs built Ethernet at Xerox PARC in the mid-1970s — after Metcalfe had spent three months with ALOHAnet.
- The bus: one coaxial cable along the building, stations tapped onto it: a broadcast LAN, run by the CSMA/CD we derived.
- The hub: by the late 1990s, twisted pairs in a star around a hub: easier to wire and repair, logically the same cable.
- The switch: in the early 2000s the hub became a switch: still a star, now store-and-forward and collision-free.
- Token ring, FDDI, ATM each did better for a while. Ethernet won by being first, cheap, and always catching up on speed.

The Ethernet frame



- Preamble, 8: seven 10101010 lock the receiver's clock; then 10101011: the two 1s say go.
- Destination, 6: first, so a switch can start forwarding before the frame has fully arrived.
- Source, 6: the adapter that put the frame on this link.
- Type, 2: whom to hand the data to: 0x0800 IPv4, 0x0806 ARP — like IP's protocol field.
- Data, 46–1500: the datagram: 1500 is the MTU, and the whole frame reaches 1518.
- CRC, 4: the CRC-32 built on polynomials, destination to data; the preamble is not covered.

Type or length



- DIX — Digital, Intel, Xerox — put a type in those two bytes; IEEE 802.3 put a length in the same place.
- Told apart by value: every type ever assigned is above 1500. Almost all traffic is DIX, which Wireshark calls Ethernet II.
- The length form survives in a few frames — the spanning-tree messages among them — with one more header inside to say what they carry.

Why short frames are padded

An ARP packet is 28 bytes. Addresses, type and CRC add 18. What goes on the wire?

A 46-byte frame would be too short to be sure of hearing a collision. The adapter adds 18 bytes of pad: the frame is 64.

- 64 bytes from destination to CRC are the 512 bits the 2τ argument demanded, written into the format.
- Hence the minimum data field of 46 bytes: $64 - 6 - 6 - 2 - 4$.
- The receiver hands up data and pad together. The protocol above knows its own length — IP from its total-length field — and discards the rest.
- The format has kept that minimum on switched links too, where nothing collides any more.

Forty years, one frame format

- The names: 10BASE-T, 100BASE-TX, 1000BASE-LX, 10GBASE-T: the speed, BASE for Ethernet-only media, then the medium.
- From 10 Mb/s to hundreds of Gb/s, over coax, copper and fibre: the physical layer changed again and again, the frame did not.
- The service: connectionless and unreliable: no handshake, no acknowledgement; a frame that fails the CRC is dropped.
- Over TCP the lost data comes back — through the same adapter, which cannot tell a retransmission from anything else.
- Speed, media, topology, even the access protocol were replaced. What is still Ethernet is the frame format.

Summary

- A frame is addressed to an interface, by a flat 48-bit MAC address that means something only on its own LAN.
- A switch forwards with a table learnt from source addresses, floods the rest, and needs a spanning tree to survive a loop.
- VLANs cut one switched network into several broadcast domains; a 4-byte tag carries the VLAN across a trunk.
- ARP turns an IP address on the same subnet into a MAC address — by broadcast, and with no authentication.
- Off the subnet the frame goes to the gateway: IP addresses last end to end, MAC addresses one hop.
- The Ethernet frame outlived every Ethernet. Next: what lies under it — the medium, and how a bit becomes a signal.

Your chat, one layer further down

```
$ sudo tcpdump -i enp3s0 -t -e -n 'port 5000'   server on .17, client here on .23
3c:52:82:4a:1f:07 > 3c:52:82:4a:2b:91, ethertype IPv4 (0x0800), length 74: ... Flags [S]
3c:52:82:4a:2b:91 > 3c:52:82:4a:1f:07, ethertype IPv4 (0x0800), length 74: ... Flags [S.]
3c:52:82:4a:1f:07 > 3c:52:82:4a:2b:91, ethertype IPv4 (0x0800), length 66: ... Flags [.]
3c:52:82:4a:1f:07 > 3c:52:82:4a:2b:91, ethertype IPv4 (0x0800), length 76: ... Flags [P.]
```

- Before the comma: the link layer: two adapters and type `0x0800`. None of it is in your code.
- Length 76: 14 of Ethernet, 20 of IP, 32 of TCP, and at byte 66 your own header: `00 00 00 05 02`, then `hello`.
- On loopback: Linux prints an Ethernet header of zeros: nothing left the machine, and the kernel is pretending.
- On WiFi: you still see Ethernet II: the driver has already translated the radio frames before the capture sees them.

Before the first SYN

Flush the ARP table, then start your client. What is the first frame on the wire?

Not the SYN. The kernel cannot build a frame for the server before it knows its MAC: an ARP exchange goes first.

```
$ sudo tcpdump -i enp3s0 -t -e -n 'arp or port 5000'
3c:52:82:4a:1f:07 > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806), length 42: Request who-has 192.168.1.17
3c:52:82:4a:2b:91 > 3c:52:82:4a:1f:07, ethertype ARP (0x0806), length 60: Reply 192.168.1.17 is-at ...
3c:52:82:4a:1f:07 > 3c:52:82:4a:2b:91, ethertype IPv4 (0x0800), length 74: ... Flags [S]
```

- The request goes to all ones; the reply comes back to your adapter alone. Then, and only then, the handshake.
- Server on another subnet? The ARP is for the gateway, and the server's MAC address never crosses your wire.

A frame shorter than the minimum

Your request went out at 42 bytes. The minimum is 64. Is your adapter breaking the rule?

No. The capture sees the frame before the adapter finishes it: 18 bytes of pad and 4 of CRC are added below where software looks.

- The reply came in at 60: padded by the other adapter, its CRC checked and removed by yours.
- No preamble, ever: a capture starts at the destination address.
- Between two virtual machines both may read 42: no real adapter touched them.
- So the one field that exists because of the speed of light is the one your tools cannot show. The 2τ rule lives in hardware.